

PointNXT MCP - Complete Technical Context

Document scope

This document describes the current repository implementation only. It does not invent backend APIs or claim unsupported capabilities.

1. Project overview

PointNXT MCP is a read-only Model Context Protocol server for retrieving PointNXT commerce data through an AI assistant. The architecture is MCP client -> server.py -> domain tools -> services/pointnxt_api.py -> PointNXT backend. It is intended for operational users working with orders, customers, products, inventory summaries, shipments, returns, and business indicators.

2. Architecture and files

server.py creates MCPServer("PointNXT MCP"), installs MCPSessionMiddleware, registers prompts, auth routes, and tools. tools/orders.py handles order queries and summaries. tools/customers.py derives customer views from orders. tools/products.py handles catalog and backend-provided inventory fields. tools/shipments.py derives shipment/fulfillment views from orders. tools/returns.py derives return views from orders. tools/business.py composes read-only business summaries. tools/auth.py handles browser and credential login. tools/system.py exposes diagnostics. services/pointnxt_api.py is the shared HTTP client. services/auth_session.py, session_store.py, request_context.py, and mcp_session_middleware.py manage authentication context. services/validation.py contains shared validation. services/metrics.py records process-local metrics. prompts.py registers reporting prompts. Dockerfile/docker-compose.yml provide deployment configuration.

3. Registered MCP tools

Authentication: current_user, auth_status, authenticate_with_pointnxt, login_with_credentials, logout_pointnxt.

Orders: get_orders, get_order_by_id, search_orders, get_order, get_orders_summary, get_order_stats, get_status_transitions.

Customers: find_customer, get_customer_context.

Products: get_products, get_product_by_id, search_products, get_product, get_product_summary.

Inventory: search_inventory, get_inventory_summary.

Shipments/fulfillment: search_shipments, get_order_shipments, get_fulfillment_queue.

Returns: search_returns, get_order_returns.

Business: get_business_summary, get_attention_queue.

System: check_backend_health, check_authentication, check_configuration, get_mcp_metrics.

All commerce tools are read-only. Login, logout, and token refresh modify authentication state only.

4. Confirmed backend endpoints

POST /auth/login; POST /auth/logout; configured refresh endpoint, default /auth/refresh-token; GET /commerce/orders; GET /commerce/orders/{order_id}; GET /commerce/orders/stats; GET /commerce/orders/status-transitions; GET /commerce/products; GET /commerce/products/{product_id}; GET /commerce/products/summary.

No dedicated customer, inventory, shipment, return, refund, or analytics endpoint is confirmed in the source.

5. Orders

get_orders is the legacy/raw paginated order query. Supported filters include order status, payment method, channel, seller, warehouse, customer ID, dateFrom/dateTo, order number/reference, customer email, customer phone, and channel order ID. Statuses are validated against the supported PointNXT order status set. Dates use YYYY-MM-DD. Core limits are positive and capped at 100.

search_orders calls the same order-list endpoint and returns compact LLM-friendly records. get_order_by_id calls the internal-ID endpoint and preserves the raw response. get_order strips a leading # from a display reference, searches the order list, then retrieves detail by returned internal ID. get_orders_summary summarizes the retrieved page with counts, status/payment breakdowns, order value, average order value, pagination, and recent orders. get_order_stats and get_status_transitions directly wrap their confirmed endpoints.

6. Customers

There is no dedicated customer API. find_customer and get_customer_context reuse GET /commerce/orders. Name, email, phone, and customer ID are passed through existing order search/filter behavior. Results are deduplicated by customer ID/contact/name. Multiple candidates are returned rather than silently selecting one. Context includes customer contact data, order history, purchased-item summaries, status counts, and returned order value. Limitations are inherited from order search and pagination; standalone customer statistics are not confirmed.

7. Products and inventory

get_products is the legacy/raw catalog query. Filters are SKU, name, status, barcode, brand, category, channel, vendor, seller, and warehouse. search_products returns compact results and can retrieve by product ID. get_product returns compact detail; get_product_by_id preserves raw detail. get_product_summary wraps the product summary endpoint. There is no dedicated inventory endpoint. search_inventory reads inventory or stocks attached to product records. get_inventory_summary returns the backend product summary. The MCP never assumes available stock equals on-hand minus reserved. If inventory fields are absent, inventory_available is false and the value is unavailable.

8. Shipments and fulfillment

No dedicated shipment API is implemented. search_shipments and get_order_shipments derive stored shipment data from order records, including shipments, warehouse, order status, and channel status. Tracking is stored backend data; no live carrier refresh is performed. get_fulfillment_queue uses PENDING orders and explicitly reports that separate picking/packing queue data is not exposed.

9. Returns and refunds

No dedicated return or refund API is implemented. search_returns uses supported order statuses RETURN_REQUESTED, RETURN_RECEIVED, PARTIALLY_RETURNED, RETURNED, REFUNDED, and PARTIALLY_REFUNDED. get_order_returns searches those statuses for an order and deduplicates results. Return ID, order, customer, status, reason, timestamps, items, quantities, inspection, putaway, stock restoration, refund status, and refund amount are returned only when backend fields exist. Quantities are not locally calculated. Refund execution is not implemented. A completed/returned status does not prove a refund was paid.

10. Business assistant

get_business_summary validates an optional UTC date and composes daily orders, order statistics, and product/inventory summary. It returns as_of, period, order count, status counts, order value, successful backend sections, and safe unavailable sections. Order value is not called profit or collected cash. get_attention_queue composes pending/on-hold/partially fulfilled daily orders, pending fulfillment, pending shipment views, and returns. Each attention category includes a reason and evidence. It does not invent arbitrary priorities, live tracking, channel issues, or inventory risks that the backend cannot support. Composite calls preserve successful sections and use safe error codes for unavailable sections.

11. Authentication flow

Credential login calls POST /auth/login with email and password. Browser login uses PointNXT login URL and state-validated callback at /auth/callback; authorization code exchange also

uses POST /auth/login. The authentication response supplies access token, refresh token, tenant ID, and optional user/membership/expiry values. API requests use Authorization: Bearer and x-tenant-id. A 401 triggers one refresh request. logout calls POST /auth/logout and clears the current session. /auth/check reports authentication state without tokens.

12. Session architecture

request_context.py stores a ContextVar request key. mcp_session_middleware.py binds each MCP request to its connection session ID. session_store.py stores AuthSession objects by key. auth_session.py reads and writes the session associated with the current key. Stateful MCP connections are isolated by connection key. Known limitation: stateless HTTP has no stable MCP session ID and currently falls back to the shared stdio key. Stateless multi-user deployment is therefore unsafe and requires a future fix. This project does not claim that fix is implemented.

13. Tenant isolation and authorization

Tenant ID comes from the authenticated PointNXT response or controlled development configuration. It is stored with AuthSession and propagated as x-tenant-id. No MCP tool accepts tenantId, so the model cannot directly choose another tenant. The MCP does not implement RBAC, membership checks, role checks, or admin checks. Authorization is delegated to PointNXT backend controls.

14. Shared API client

PointNXTAPI supports synchronous and asynchronous GET, POST, PUT, and DELETE. Read tools use async GET. Requests use JSON headers, bearer authentication, tenant propagation, request IDs, latency metrics, structured logs, timeout handling, and retries for timeout/connection/502/503/504. A 401 performs one refresh and retries once. PII-like parameter values, including generic search, are fingerprinted before logging. Tokens and passwords are not logged.

15. Validation

services/validation.py provides page_limit with maximum 100, optional_text with maximum 200 characters, optional_id, and optional_date. Orders and products also retain domain validation. Customer, shipment, return, and business tools use shared validators. Status validation is domain-specific. Boolean values are rejected for numeric pagination where the validator is applied.

16. Errors and security

The client normalizes timeout, connection, HTTP, refresh, and validation failures. No-result responses are explicit. Customer ambiguity returns candidates. Business composite tools return successful sections and unavailable entries with safe error codes: backend_unavailable, authentication_required, request_timeout, and section_failed. Request logs retain timestamp, request ID, method, endpoint, status, attempts, latency, and tenant metadata while fingerprinting sensitive parameters.

17. Tests

test_orders.py covers order validation, retrieval, summaries, stats, transitions, and failures. test_customers.py covers deduplication, context, and ambiguity. test_products.py covers filters, pagination, lookup, summary, and default pagination. test_inventory.py covers inventory views and summaries. test_shipments.py covers stored shipment and fulfillment limitations. test_returns.py covers return statuses/items and refund non-execution. test_business.py covers business summary and attention behavior. test_security.py covers session/tenant helper isolation, safe logging parameters, partial failures, and registered tool names.

Remaining test gaps include full stateful HTTP integration, safe stateless HTTP behavior, backend RBAC, real backend contract tests, and end-to-end browser callback testing.

18. Deployment

Dockerfile and docker-compose.yml run the server on port 8000. Configuration comes from .env: POINTNXT_BASE_URL, POINTNXT_LOGIN_URL, POINTNXT_AUTH_CALLBACK_URL, POINTNXT_ACCESS_TOKEN, POINTNXT_REFRESH_TOKEN, POINTNXT_REFRESH_ENDPOINT, POINTNXT_TENANT_ID, DEV_MODE, and AUTH_CALLBACK_TIMEOUT. The documented HTTP startup is server.mcp.run('streamable-http', host='0.0.0.0', port=8000). External URL and proxy configuration are not fixed by source code.

19. Current limitations and readiness

Implemented: read-only order/product querying, derived customer/shipment/return views, backend inventory summaries, authentication and refresh, tenant header propagation, diagnostics, compact business tools, validation, safe parameter logging, tests, and Docker deployment.

Partially implemented/backend-dependent: customer data, shipment detail, returns, inspection/putaway, stock semantics, analytics, authorization.

Not implemented: commerce mutations, dedicated customer/inventory/shipment/return/refund APIs, RBAC, live tracking refresh, arbitrary priority ranking.

Ready for controlled office testing with trusted users and stateful sessions. Stateful multi-user production still needs end-to-end transport isolation tests and backend authorization confirmation. Stateless multi-user deployment is not ready because of the shared fallback session key.

20. Design decisions

Existing backend services are reused to avoid duplicate business logic. Customer, shipment, and return features are derived from orders because dedicated APIs are not confirmed.

Inventory is not calculated locally because backend semantics must control sellable stock. Raw and compact tools coexist for backward compatibility. Write tools were intentionally not added. tenantId is kept out of MCP parameters to prevent model-controlled tenant switching.

21. Example interactions

Find order #1503: search_orders(order_reference='1503'), then get_order(order_reference='#1503') for detail. Find Krishna Saraf: find_customer(name='Krishna Saraf'), then get_customer_context with a unique candidate. Products matching Laptop: search_products(name='Laptop'). Track order: get_order_shipments(order_reference='#1503'), which reports stored data only. Returns: get_order_returns(order_reference='#1503'). Daily summary: get_business_summary(). Attention: get_attention_queue().

22. Quick reference

Important files: server.py; tools/orders.py, customers.py, products.py, shipments.py, returns.py, business.py, auth.py; services/pointnxt_api.py, auth_session.py, session_store.py, request_context.py, mcp_session_middleware.py, validation.py, metrics.py. Confirmed endpoints are auth login/logout/refresh plus commerce orders/products and their stats/summary/status metadata. Main risks are stateless HTTP session fallback, backend-dependent authorization, and missing dedicated domain APIs.